

Jugo web application

Technical Manual

1. System Architecture.....	3
2. Frontend Stack.....	3
3. Backend Services.....	4
4. AI / Image Analysis Pipeline.....	4
5. Data Flow Diagram (Text Version).....	4
6. Image Processing Pipeline.....	5
6.1 How Dashboards Are Analyzed.....	5
6.2 Chart Detection Logic.....	5
6.3 IBCS Rule Evaluation Logic.....	5
6.4 Confidence Scoring.....	6
7. AI / Rule Engine.....	7
7.1 Which IBCS Rules Are Automated.....	7
7.2 Rule Categorization.....	7
7.3 Threshold Logic.....	7
7.4 Limitations and Edge Cases.....	7
8. APIs & Integration.....	8
8.1 Upload Endpoints.....	8
8.2 Authentication.....	8
8.3 Response Formats.....	8
8.4 Error Handling.....	8
9. Camera Capture Implementation.....	9
9.1 Browser APIs Used.....	9
9.2 Permissions Handling.....	9
9.3 Image Preprocessing.....	9
10. Deployment & Operations.....	9
10.1 Hosting Environment.....	9
10.2 Environment Variables.....	9
10.3 Scaling Considerations.....	10
10.4 Monitoring & Logging.....	10
11. Future improvements.....	10
12. Security.....	11

1. System Architecture

The system is a simple client–server setup.

- A **web frontend** lets users upload dashboard images.
- A **backend API** receives the image and runs analysis.
- An **AI + rule engine** checks IBCS compliance based on the “starts at 0 rule”.
- The backend sends a clear result back to the frontend.

Overall flow:

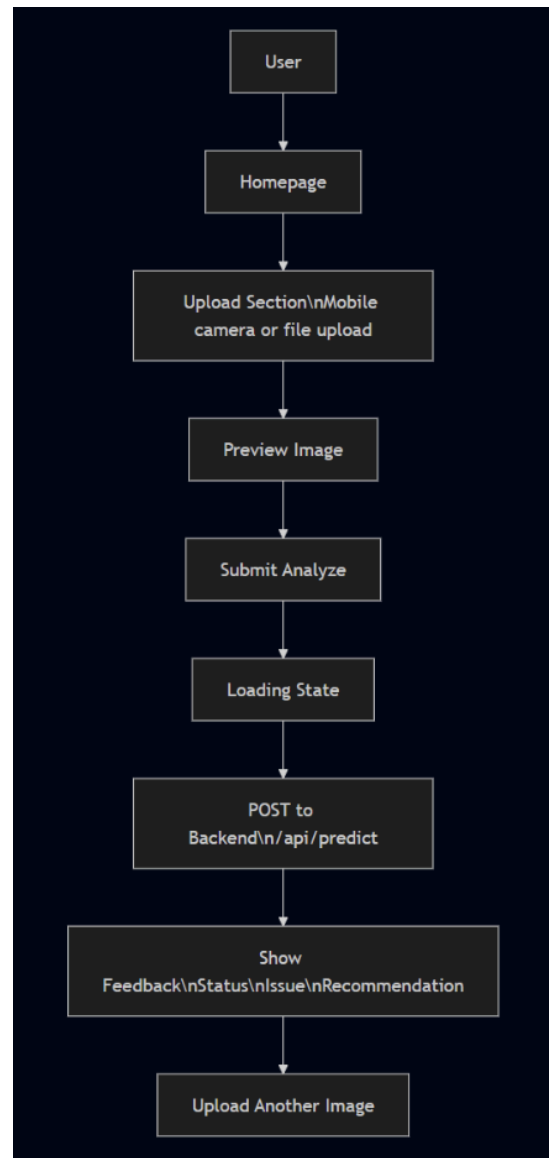
User → Browser → Backend API → AI & Rules → Result → Browser

2. Frontend Stack

The frontend is lightweight and focuses on image upload.

- Runs in the browser
- Uses HTML, CSS, and JavaScript
- Sends images to the backend using HTTP POST
- Displays compliance result and message

A visualization of how it works:



3. Backend Services

The backend is built with **FastAPI (Python)**.

Main responsibilities:

- Accept image uploads
- Validate file type
- Preprocess images
- Run AI model
- Apply IBCS rules
- Return structured JSON responses

Key file:

- `main.py` – main API service

4. AI / Image Analysis Pipeline

The AI pipeline checks if a dashboard looks compliant.

Steps:

1. Image is uploaded
2. Image is converted to RGB
3. Image is resized and normalized
4. Pretrained CNN model (MobileNet-based) runs prediction
5. Prediction score is passed to rule logic

The AI focuses on **visual structure and layout quality**.

5. Data Flow Diagram (Text Version)

1. User uploads image
2. Frontend sends image to `/api/predict`
3. Backend reads image bytes
4. Image preprocessing happens
5. AI model runs inference
6. Rule engine evaluates result

7. Compliance decision is returned

6. Image Processing Pipeline

6.1 How Dashboards Are Analyzed

- Dashboards are treated as static images
- No raw data is extracted
- Analysis is visual only

The model looks for:

- Layout consistency
- Clean structure
- Readability

6.2 Chart Detection Logic

Chart detection is **implicit**, not object-based.

- No bounding boxes
- No chart-type classification
- The CNN learns visual patterns of dashboards

This keeps the system fast and simple.

6.3 IBCS Rule Evaluation Logic

Rules are applied **after AI prediction**.

Example logic:

- High confidence score → compliant
- Low confidence score → non-compliant

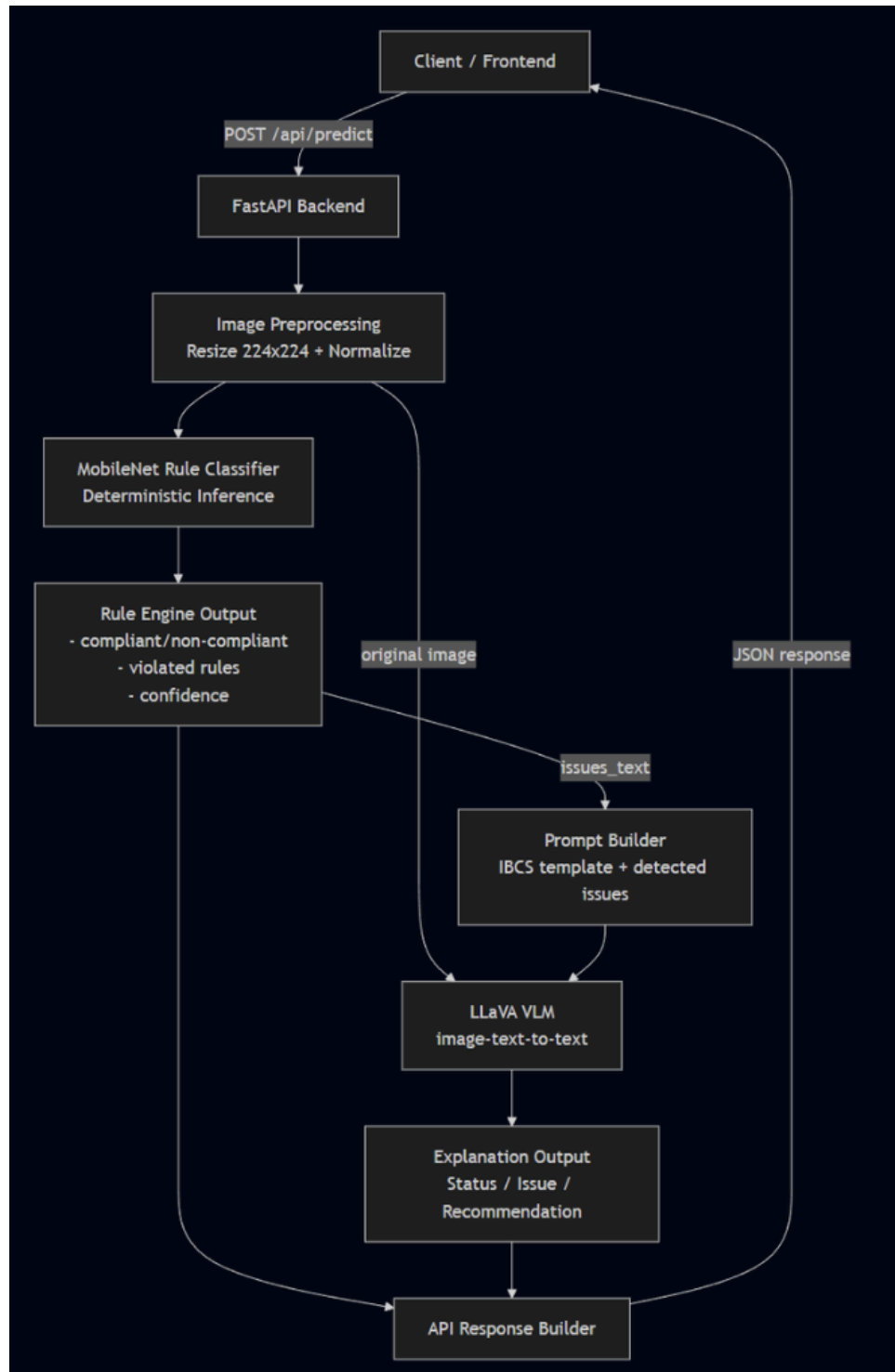
Rules are currently **binary** (pass / fail).

6.4 Confidence Scoring

- AI outputs a probability score
- Score represents visual compliance confidence
- Threshold is used to decide final result

Example:

- Score $\geq$ threshold $\rightarrow$ Compliant
- Score $<$ threshold $\rightarrow$ Non-compliant



7. AI / Rule Engine

7.1 Which IBCS Rules Are Automated

Automated (high-level):

- Visual clarity
- Starts at 0 rule

Not automated:

- Consistent_scales
- Scale breaks
- Labeled units
- Misleading axes

7.2 Rule Categorization

Rules are grouped into:

- Visual rules (AI-based)
- Decision rules (threshold-based)

7.3 Threshold Logic

- A fixed confidence threshold is used
- Threshold is defined in backend logic
- Can be adjusted during tuning

7.4 Limitations and Edge Cases

Limitations:

- Works only on images
- Sensitive to very low image quality
- Cannot understand chart data

Edge cases:

- Hand-drawn dashboards

- Extremely dense visuals
- Non-standard chart designs

8. APIs & Integration

8.1 Upload Endpoints

- `POST /api/predict`

Input:

- Multipart image file

8.2 Authentication

- No authentication (prototype stage)
- Open endpoint

8.3 Response Formats

JSON response:

```
{  
  "compliance": "Compliant",  
  "message": "Layout, readability, and structure appear suitable."  
}
```

8.4 Error Handling

Handled errors:

- Non-image upload → 400
- Invalid image → 400
- Server error → 500

9. Camera Capture Implementation

9.1 Browser APIs Used

- `getUserMedia()` (optional)
- `<input type="file" accept="image/*">`

9.2 Permissions Handling

- Browser handles camera permission
- User must approve access

9.3 Image Preprocessing

Before upload:

- Image is captured or selected
- Sent as-is to backend

Backend preprocessing:

- Resize
- Normalize
- RGB conversion

10. Deployment & Operations

10.1 Hosting Environment

- Backend runs as Python service
- Can be deployed on cloud VM or container

10.2 Environment Variables

Typical variables:

- Model path
- Port number
- Debug mode

10.3 Scaling Considerations

- Stateless API
- Horizontal scaling possible
- GPU improves performance but not required

10.4 Monitoring & Logging

- FastAPI logs requests
- Errors logged to console
- Can be extended with monitoring tools

11. Future improvements

Currently not implemented.

Future use:

1. Extend the amount of rules the model checks to:
 - 'start_at_zero'
 - 'consistent_scales'
 - 'scale_breaks'
 - 'labeled_units'
 - 'Misleading_axes'

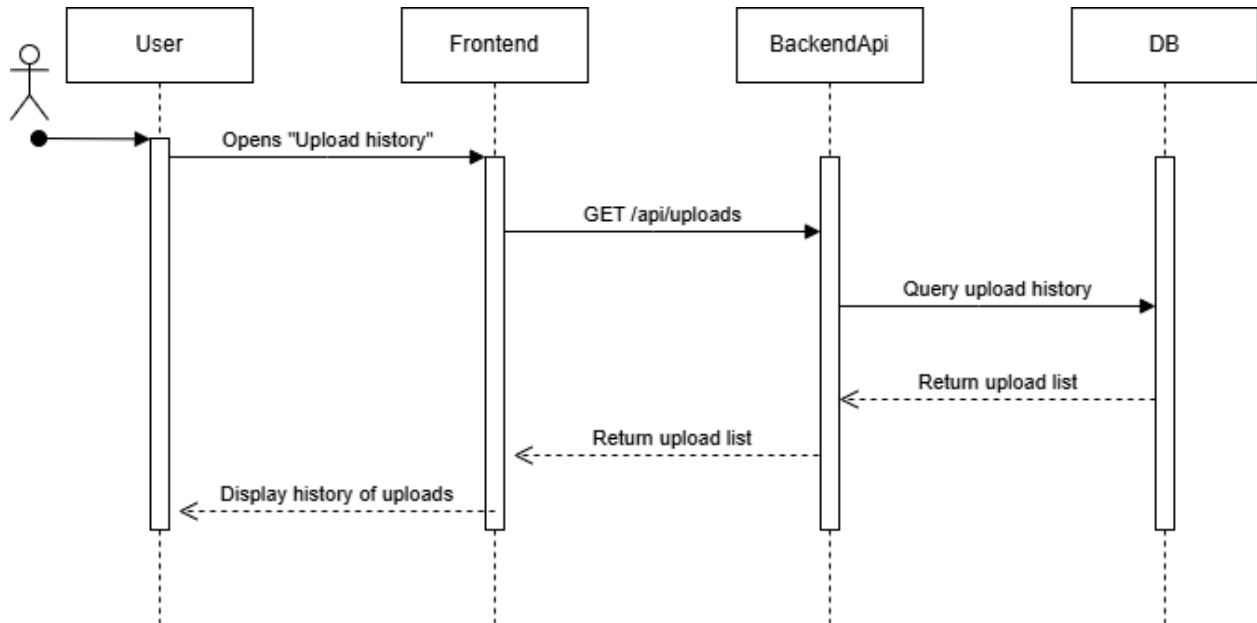
This would require a dataset that consists of dashboard images with the corresponding labelled rule it breaks.

2. A useful future improvement would be to store a history of uploaded dashboards and their analysis results, allowing users to:
 - view previous uploads and compliance outcomes,
 - compare changes over time,
 - re-download reports,

- track improvement progress for IBCS compliance.

This feature was out of scope for the current prototype because the system is currently stateless and does not use a database or persistent storage.

The process would look like this:



12. Security

The current prototype does not implement user accounts or persistent storage and is therefore fully stateless. As a result, no authentication or authorisation mechanisms are currently required.

However, if the system is extended with user accounts and upload history stored in a database, security becomes an essential requirement.

In a future version of the system, the following security measures should be implemented:

1. User Authentication and Authorisation

Users should be required to authenticate before using the system. Each user should only be able to access their own uploaded dashboards and analysis results. This prevents unauthorised access to sensitive or private business dashboards.

2. Protected API Endpoints

Certain API endpoints, such as those used to retrieve upload history, system statistics, or administrative information, should be protected and only accessible to authorised users or administrators.

End of Technical Manual